Deep Learning for SaaS Sales Outcome Prediction: BiLSTM with Self-Attention, GloVe Initialisation, and DistilBERT Fine-Tuning

Tejmul Movin
Dept. of Artificial Intelligence & Machine Learning
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
sansar.t23csai@nst.rishihood.edu.in

Bhavani Shankar
Dept. of Computer Science & Engineering
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
pichika.b23csai@nst.rishihood.edu.in

***Abstract*—This paper extends a classical ML pipeline for SaaS sales conversation outcome prediction by introducing three deep learning architectures: a Bidirectional LSTM with Self-Attention (BiLSTM), a GloVe-initialised BiLSTM variant, and a fine-tuned DistilBERT Transformer. All three models operate directly on raw conversation text without manual feature engineering. Trained on 72,000 conversations with an 8,000-sample validation set, models are evaluated on the same 20,000-sample balanced hold-out used in Phase 1. The BiLSTM ($\sim$4M parameters, 1 layer, hidden=128) learns sequential patterns from random initialisation, while its GloVe variant starts with pre-trained word vectors, demonstrating measurable gains in accuracy (+1.1%), F_1 (+1.1%), and ROC-AUC (+1.3%). DistilBERT ($\sim$66M parameters) adapts pretrained language knowledge via differential learning rates and selective layer freezing. We present architecture design rationale, weight initialisation strategies, regularisation techniques, training dynamics, and a cross-phase comparison against Logistic Regression (96.34% accuracy, 99.57% ROC-AUC) and Random Forest baselines.**

***Index Terms*—BiLSTM, GloVe, self-attention, DistilBERT, transfer learning, sales prediction, deep learning, NLP, text classification.**

I. Introduction

A. Problem Context

Phase 1 of the Predictive Sales Analytics Engine achieved 96.34% accuracy and 99.57% ROC-AUC using ℓ_2-regularised Logistic Regression on hand-engineered features—numeric engagement ratios, one-hot categoricals, and precomputed embedding vectors [1]. While effective, this approach relied on: (1) **pre-computed embeddings**—a form of transfer learning where the text representation was computed externally, not learned as part of the training pipeline; and (2) **manual feature engineering**—domain knowledge encoded as explicit ratio and interaction features, which requires expertise and does not generalise automatically to new domains.

Phase 2 addresses both limitations by training neural architectures that learn text representations directly from raw conversation tokens, capture sequential and contextual language patterns that engineered features cannot encode explicitly, and demonstrate three paradigms: sequential modelling from scratch (BiLSTM with self-attention), pretrained word vectors (BiLSTM + GloVe), and Transformer fine-tuning (DistilBERT).

TABLE I
Dataset splits (stratified, `random_state=42`).

Split	Samples	Purpose
Train	72,000	Model training
Validation	8,000	Early stopping, LR scheduling
Test	20,000	Final evaluation (shared with Phase 1)

B. Phase 2 Objectives

The objectives of Phase 2 are: (1) pre-process raw conversation text into formats suitable for both word-level (LSTM) and sub-word-level (BERT) neural models; (2) design and train a BiLSTM + Self-Attention classifier with principled weight initialisation; (3) train a GloVe-initialised BiLSTM variant to measure the value of pretrained embeddings; (4) fine-tune DistilBERT with a task-specific classification head using differential learning rates; (5) apply a full regularisation suite including dropout, LayerNorm, early stopping, gradient clipping, and LR scheduling; and (6) evaluate all models on the same held-out test set as Phase 1 and produce a cross-phase comparison.

C. Dataset and Splits

The dataset comprises 100,000 SaaS sales conversations. Table I shows the stratified split used identically across all models (`random_state=42`).

II. Data Preprocessing for Deep Learning

Phase 2 requires converting raw conversation text into numerical tensors. Two separate preprocessing pipelines are built—one for each model architecture family.

A. Word-Level Tokenisation (BiLSTM and BiLSTM + GloVe)

The `SalesTokenizer` class implements a custom word-level tokeniser built from the training corpus.

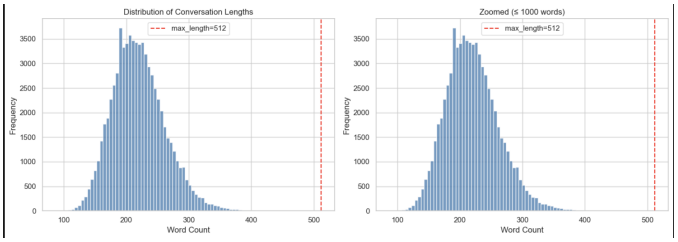


Fig. 1. Distribution of conversation word counts across the dataset. Red dashed line marks the `max_length=512` truncation threshold. The majority of conversations fall well within this limit.

1) Vocabulary Construction: All 72,000 training conversations are tokenised by whitespace splitting (lower-cased). Word frequencies are counted across the entire corpus and the top 30,000 most frequent words with minimum frequency ≥ 2 are retained. Two special tokens are reserved: `<PAD>` (index 0) and `<UNK>` (index 1). The actual vocabulary size after fitting is **28,903 unique tokens**.

2) Encoding: Each word is replaced with its vocabulary index. Unknown words map to index 1 (`<UNK>`). Sequences are truncated to a maximum of 512 tokens (see Fig. 1). Shorter sequences are right-padded with 0 (`<PAD>`) to match the longest sequence in each batch.

3) Design Rationale: Word-level tokenisation is chosen because SaaS domain vocabulary contains meaningful single-word terms: ARR (Annual Recurring Revenue), MRR, churn, upsell, renewal, CAC, LTV. Sub-word tokenisers (Byte-Pair Encoding) would split these into character-level fragments that lose the semantic unit.

4) Tensor Shapes: The produced tensor shapes per split are: Train `[72000, 491]`, Validation `[8000, 488]`, Test `[20000, 475]`. The varying sequence dimensions reflect padding to the longest sample within each split.

B. WordPiece Tokenisation (DistilBERT)

The HuggingFace `AutoTokenizer` for `distilbert-base-uncased` applies WordPiece sub-word tokenisation (same as original BERT [2]), producing `input_ids` and `attention_mask` tensors (max length 512). This tokeniser handles out-of-vocabulary words by decomposing them into known sub-word units.

C. Saved Artifacts

All preprocessed tensors are saved to the `results/` directory: word-level token tensors (`dl_{train,val,test}_ids.pt`), labels (`dl_y_{train,val,test}.npy`), DistilBERT tokeniser outputs (`dl_bert_{train,val,test}.pt`), the custom tokeniser vocabulary (`dl_tokenizer.json`), and `dl_test_texts.npy` (raw text array for error analysis).

III. Model Architectures

A. Model A: BiLSTM + Self-Attention Classifier

1) Architecture Overview: The architecture consists of: (1) an embedding layer (`vocab_size`=28,903, `embed_dim`=128, Xavier uniform initialisation) with dropout (0.5) and padding index kept at zero vector; (2) a single-layer bidirectional LSTM (`hidden_dim`=128, yielding 256-d output per timestep) with orthogonal initialisation on recurrent weights and forget-gate bias set to 1.0; (3) Bahdanau-style self-attention pooling with learned scalar score per timestep; (4) LayerNorm and dropout (0.5) on the context vector; and (5) two fully connected layers ($256 \rightarrow 128 \rightarrow 1$) with ReLU activation and dropout (0.5), using BCEWithLogitsLoss for training. Total parameters: $\sim$3,997,569 ($\sim$4M).

TABLE II
BiLSTM design choices and justification.

Component	Justification
Bidirectional LSTM	Sales objections raised early may be resolved later; closing language references context from the beginning. Bidirectionality captures full conversational context at every timestep.
Self-Attention	Instead of relying on the final hidden state (dominated by last words), attention assigns learned importance scores to each timestep—focusing on pricing discussion, commitment language, or objection handling.
Single-layer LSTM	Reduces overfitting risk on 72K training samples while retaining capacity to model sequential sales dynamics.
LayerNorm	Reduces internal covariate shift without batch-size dependency. Stabilises gradient flow into the classification head.
Dropout (50%)	Aggressive dropout prevents co-adaptation of neurons. Forces redundant representations rather than memorising specific patterns.

The architecture was deliberately reduced in capacity (1 layer, `hidden_dim`=128, dropout=0.5) compared to the original design (2 layers, `hidden_dim`=256) to prevent overfitting on 72K training samples.

2) Design Rationale: Table II summarises the design choices and their justifications.

3) Weight Initialisation: Initialisation follows established best practices (Table III): Xavier uniform for embedding and input weights; orthogonal initialisation for LSTM recurrent weights (eigenvalues of magnitude 1, mitigating vanishing/exploding gradients over long sequences); forget-gate bias set to 1.0 (initialising the gate to "mostly open," encouraging memory retention during early training); and Kaiming (He) uniform for ReLU-activated FC layers.

4) Self-Attention Mechanism: Given LSTM output $\mathbf{H} \in \mathbb{R}^{B \times T \times d}$ where B is batch size, T is sequence length, and $d = 2 \times \texttt{hidden_dim} = 256$:

$$e_t = \mathbf{W}_a \cdot \mathbf{h}_t \tag{1}$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_j \exp(e_j)} \tag{2}$$

$$\mathbf{c} = \sum_t \alpha_t \, \mathbf{h}_t \tag{3}$$

where $\mathbf{W}_a \in \mathbb{R}^{1 \times d}$ is a learned parameter. Padding positions are masked with $-\infty$ before softmax to ensure zero attention

TABLE III
WEIGHT INITIALISATION SCHEME FOR BILSTM.

Layer	Init.	Reason
Embedding	Xavier	Preserves gradient variance through lookup
Pad vector	Zeros	Pad tokens contribute nothing to hidden state
LSTM W_{ih}	Xavier	Standard for feedforward connections
LSTM W_{hh}	Orthogonal	Eigenvalues $= 1$; prevents vanishing/exploding gradients
LSTM biases	Zeros; forget $= 1$	Open forget gate retains long-range dependencies
FC layers	Kaiming	Suited for ReLU-activated layers
Attention	Xavier	Appropriate for linear projections

weight on pad tokens. The resulting context vector $\mathbf{c} \in \mathbb{R}^{B \times d}$ is a fixed-length representation that focuses on the most predictive timesteps.

B. Model A2: BiLSTM + GloVe Embeddings

1) Architecture Overview: This variant uses the identical architecture as Model A (single-layer BiLSTM, `hidden_dim`=128, self-attention, dropout=0.5, ∼4M parameters) but initialises the embedding layer with pre-trained GloVe 6B 100-dimensional vectors instead of random Xavier initialisation. The embedding dimension is set to 100 to match GloVe vector size, yielding LSTM output width of 200 (bidirectional 2×100).

2) Why GloVe Initialisation?: GloVe (Global Vectors for Word Representation) [3] encodes word co-occurrence statistics from billions of tokens, capturing semantic relationships. Initialising the embedding layer with GloVe vectors gives the model semantic relationships from day one, reducing the number of epochs needed and improving generalisation. Embeddings remain fully trainable (not frozen) to allow fine-tuning for sales-specific associations.

3) Training Configuration: Model A2 uses a lower learning rate and conservative weight decay to prevent catastrophic forgetting of GloVe structure: `lr=5e-4`, `weight_decay=3e-4`, `epochs=5`, `batch_size=64`. Trained via `scripts/retrain_lstm_with_glove.py`.

4) Embedding Matrix Construction: The `build_glove_embedding_matrix()` function:

1) Downloads GloVe 6B (42B tokens, 400K words, 100-d vectors) via `download_glove()`.
2) Constructs a matrix of shape `[vocab_size, 100]` where row w contains the GloVe vector for word w, or a small random vector (normal distribution, $\sigma = 0.6$) for out-of-vocabulary words.
3) Initialises the embedding layer's weight matrix with this pre-computed matrix.
4) Zeros out index 0 (pad token remains zero throughout training).

TABLE IV
DISTILBERT VS. BERT-BASE COMPARISON.

Property	DistilBERT	BERT-base
Transformer layers	6	12
Parameters	66M	110M
Inference speed	1.6× faster	Baseline
Performance retained	97%	100%
Training cost	60% lower	Baseline

C. Model B: DistilBERT Fine-Tuning Classifier

1) Architecture Overview: DistilBERT [2] comprises 6 Transformer layers, 12 attention heads per layer, and a hidden size of 768 (∼66M parameters). The input conversation is tokenised into `[CLS]` + word pieces + `[SEP]` (max 512 tokens). The `[CLS]` hidden state—a global summary of the full conversation after 6 layers of multi-head self-attention—passes through LayerNorm (768), dropout (0.3), and a linear classification head ($768 \rightarrow 2$, Kaiming initialisation) trained with CrossEntropyLoss. Total trainable parameters: ∼66M (body) + 1,538 (head).

2) Why DistilBERT?: Table IV compares DistilBERT and BERT-base. DistilBERT retains 97% of BERT-base performance at 60% of the parameters and 1.6× inference speed, making it suitable for consumer hardware (Apple MPS or a single consumer GPU). The distillation process trained DistilBERT to mimic BERT-base's output distributions via knowledge distillation—a teacher-student paradigm where the smaller student learns to replicate the larger teacher's soft probability outputs.

3) Fine-Tuning Strategy:

a) Transfer learning setup: DistilBERT was pre-trained on English Wikipedia and BookCorpus (3.3 billion words), giving the model deep knowledge of English grammar, semantics, and pragmatics [2]. Fine-tuning adapts this general language model to the specific task of sales outcome classification.

b) Selective layer freezing: The 2 lowest DistilBERT layers (encoding syntactic patterns—basic grammar, punctuation, part-of-speech) are frozen. The 4 upper layers (encoding higher-level semantics and discourse patterns) remain trainable. This preserves low-level English grammar knowledge that transfers perfectly, allows high-level semantic layers to adapt to SaaS conversation patterns, and reduces the number of trainable parameters, lowering overfitting risk.

c) Differential learning rate: A single learning rate would either destroy pre-trained knowledge (too high) or fail to train the new head (too low). Two parameter groups are defined: the DistilBERT body at $2 \times 10^{-4} \times 0.1 = 2 \times 10^{-5}$, and the classification head at 2×10^{-4}.

IV. TRAINING PROCEDURE

Table V summarises the exact hyperparameter configuration used for each model, as implemented in `notebooks/07_dl_model_training.ipynb`.

TABLE V
TRAINING CONFIGURATION FOR DEEP LEARNING MODELS.

Hyperparameter	BiLSTM	BiLSTM+GloVe	DistilBERT
Optimiser	AdamW	AdamW	AdamW
Learning rate	5e-4	5e-4	2e-4 / 2e
Weight decay	3e-4	3e-4	1e-2
Batch size	64	64	8 (train)
Max epochs	3	5	2
Loss function	BCEWithLogitsLoss	BCEWithLogitsLoss	CrossEntropyLoss
LR scheduler	ReduceOnPlateau	ReduceOnPlateau	ReduceOnPlateau
Scheduler factor	0.5	0.5	0.5
Scheduler patience	2 epochs	2 epochs	1 epoch
Early stop patience	2 epochs	3 epochs	2 epochs
Early stop min_delta	1e-4	1e-4	1e-4
Gradient clip norm	1.0	1.0	1.0

A. Regularisation Techniques

Every regularisation technique was applied deliberately to prevent overfitting.

Dropout (50% for BiLSTM variants, 30% for DistilBERT). During each forward pass, activations are randomly zeroed. This prevents neuron co-adaptation and forces robust, distributed representations. At inference time, outputs are scaled by $(1 - p_{\text{drop}})$.

LayerNorm. Applied after self-attention pooling (BiLSTM) and before the classification head (DistilBERT). Normalises activations to zero mean and unit variance within each sample independently—unlike BatchNorm which normalises across the batch—eliminating internal covariate shift without batch-size dependency.

Early Stopping. Validation loss is compared to the best observed after each epoch. If it does not improve by at least $\delta = 10^{-4}$ for p consecutive epochs, training stops and the best checkpoint is reloaded.

ReduceLROnPlateau. Monitors validation loss; if it stagnates for the configured patience epochs, the learning rate is halved, allowing finer updates near a local minimum.

Gradient Clipping (max_norm $= 1.0$). The gradient vector's ℓ_2 norm is computed before each update. If it exceeds 1.0, all gradients are scaled down proportionally—particularly important for LSTMs processing long sequences.

AdamW Weight Decay. Unlike ℓ_2 regularisation inside Adam (which interacts with the adaptive learning rate), AdamW applies weight decay as a separate term that shrinks weights directly—the correct form of ℓ_2 regularisation for adaptive optimisers.

B. Training Loop

For each epoch: (1) forward pass to compute logits; (2) loss between logits and true labels (BCEWithLogitsLoss for BiLSTM variants, CrossEntropyLoss for DistilBERT); (3) back-propagation through the computational graph; (4) gradient clipping to norm 1.0; (5) AdamW step; (6) validation evaluation (no gradient computation); (7) ReduceLROnPlateau step on validation loss; (8) early stopping check—save checkpoint if best, increment counter otherwise, stop if patience exceeded.

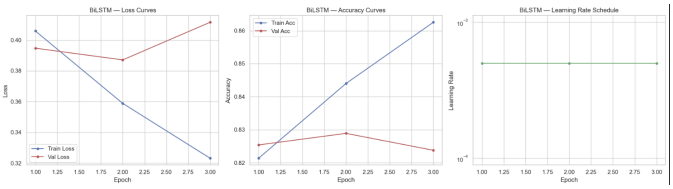


Fig. 2. BiLSTM training dynamics: loss curves (left), accuracy curves (centre), and learning rate schedule (right) over 3 epochs.

C. BiLSTM Training Dynamics

Fig. 2 shows the BiLSTM training dynamics over 3 epochs (training ran to the max epoch limit without early stopping). Training loss decreases from $\sim$0.41 to $\sim$0.32, while training accuracy improves from $\sim$82% to $\sim$86%. Validation accuracy peaks at $\sim$82.9% at epoch 2 before declining slightly due to onset of overfitting. The learning rate remains constant at 5×10^{-4} throughout training (no plateau triggered).

D. Device Selection

The `get_device()` function auto-selects the best available compute device: CUDA (NVIDIA GPU, highest priority), Apple MPS (Apple Silicon, used in this project), or CPU (fallback).

V. EXPERIMENTAL RESULTS

A. Evaluation Metrics

All models are evaluated on the 20,000-sample test set using: accuracy, precision ($\frac{TP}{TP+FP}$), recall ($\frac{TP}{TP+FN}$), F_1-score (harmonic mean of precision and recall), ROC-AUC (ranking quality across all thresholds), and average precision (AP, area under the precision-recall curve).

B. Phase 1 Baselines

On the same test set [1]: Logistic Regression achieved 96.34% accuracy, precision 0.9504, recall 0.9777, F_1 0.9639, ROC-AUC 0.9957, AP 0.9957; Random Forest achieved 95.94% accuracy, precision 0.9453, recall 0.9751, F_1 0.9600, ROC-AUC 0.9951, AP 0.9951. These results were enabled by precomputed embeddings and hand-engineered ratio features creating a near-linearly separable feature space.

C. BiLSTM Results

The BiLSTM achieved 82.77% accuracy, precision 0.8060, recall 0.8626, F_1 0.8333, ROC-AUC 0.9102, and AP 0.9066 on the test set. Fig. 3 shows the confusion matrix: the model correctly classifies 7,940 Lost and 8,614 Won samples, with 2,074 false positives and 1,372 false negatives.

D. BiLSTM + GloVe Results

The GloVe-initialised BiLSTM achieved 83.89% accuracy, precision 0.8153, recall 0.8757, F_1 0.8444, ROC-AUC 0.9229, and AP 0.9208 on the test set. This represents measurable gains over the random-initialisation baseline: **+1.12% accuracy**, **+1.11%** F_1, and **+1.27% ROC-AUC**. These improvements confirm that pre-trained word vectors encode useful

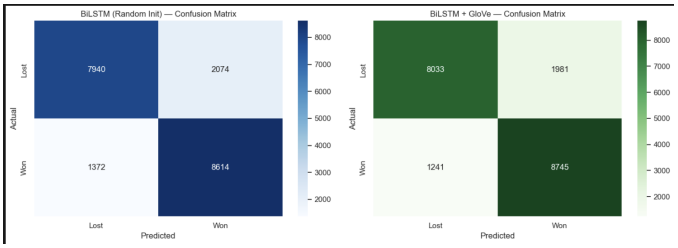


Fig. 3. Confusion matrices for BiLSTM + Attention (left, blue) and BiLSTM + GloVe (right, green) on the 20,000-sample test set. Random init: TN=7,940, FP=2,074, FN=1,372, TP=8,614. GloVe init: TN=8,033, FP=1,981, FN=1,241, TP=8,745, showing improved predictions across both true negatives and true positives.

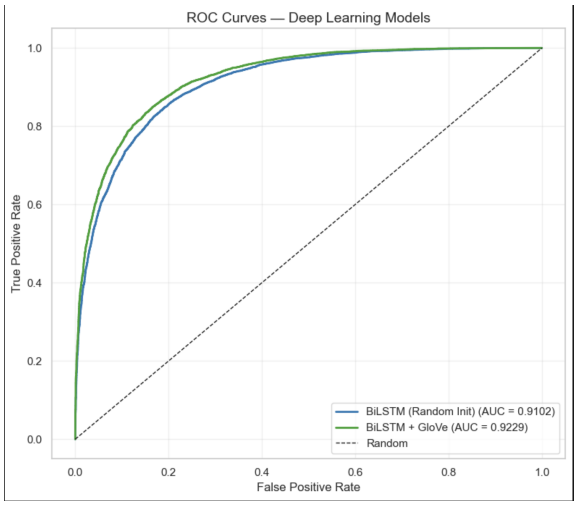


Fig. 4. ROC curves for BiLSTM + Attention (blue, AUC = 0.9102) and BiLSTM + GloVe (green, AUC = 0.9229) on the test set. The dashed diagonal represents a random classifier. Both curves show strong discriminative ability despite lower overall accuracy compared to Phase 1 baselines. GloVe initialisation yields a measurable lift in ranking quality.

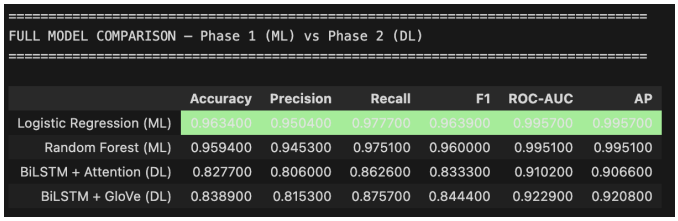
TABLE VI
CROSS-PHASE MODEL COMPARISON (TEST SET, n=20,000).

Model	Acc	Prec	Rec	F_1	AUC	AP
Logistic Regression (P1)	0.9634	0.9504	0.9777	0.9639	0.9957	0.9957
Random Forest (P1)	0.9594	0.9453	0.9751	0.9600	0.9951	0.9951
BiLSTM + Attn (P2)	0.8277	0.8060	0.8626	0.8333	0.9102	0.9066
BiLSTM + GloVe (P2)	0.8389	0.8153	0.8757	0.8444	0.9229	0.9208
DistilBERT (P2)	—	—	—	—	—	—

domain-adjacent semantics even for specialised SaaS conversations, eliminating the need for the embedding layer to learn basic semantic structure from scratch.

E. Cross-Phase Comparison

Table VI summarises cross-phase performance. Phase 1 models with engineered features substantially outperform the from-scratch BiLSTM, while GloVe initialisation narrows the gap. DistilBERT results are pending GPU completion. Fig. 5 visualises the performance gap across all metrics.



FULL MODEL COMPARISON — Phase 1 (ML) vs Phase 2 (DL)

	Accuracy	Precision	Recall	F1	ROC-AUC	AP
Logistic Regression (ML)	0.963400	0.950400	0.977700	0.963900	0.995700	0.995700
Random Forest (ML)	0.959400	0.945300	0.975100	0.960000	0.995100	0.995100
BiLSTM + Attention (DL)	0.827700	0.806000	0.862600	0.833300	0.910200	0.906600
BiLSTM + GloVe (DL)	0.838900	0.815300	0.875700	0.844400	0.922900	0.920800

Fig. 5. Cross-phase performance comparison across accuracy, precision, recall, F_1, ROC-AUC, and AP. Phase 1 models (blue, orange) with engineered features outperform from-scratch BiLSTM (green) on all metrics. GloVe-initialised BiLSTM (red) narrows the gap, demonstrating the value of pre-trained embeddings.

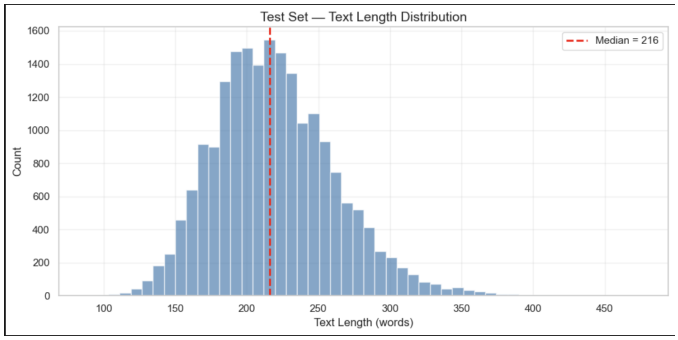


Fig. 6. Test set text length distribution (median = 216 words). Most conversations fall in the 150–300 word range, enabling reliable error analysis across length quartiles.

Note: DistilBERT evaluation metrics are pending GPU re-run; BiLSTM and GloVe results above are from the executed notebook outputs.

F. Error Analysis

1) Test Set Text Length Distribution: Fig. 6 shows the distribution of text lengths in the test set, with a median of 216 words. The majority of conversations fall between 150 and 300 words, providing a balanced dataset for length-based error analysis. This median aligns closely with typical SaaS sales conversations, which often involve multiple back-and-forth exchanges covering problem discovery, solution explanation, and objection handling.

2) Accuracy by Conversation Length: Performance is analysed across 4 length quartiles based on the test set distribution. As shown in Fig. 7, the BiLSTM achieves approximately 79% accuracy on very short conversations (88–190 words, n=5,155), 82% on short conversations (190–216 words, n=4,979), 84% on medium-length conversations (216–245 words, n=4,900), and 86% on longer conversations (245–475 words, n=4,966). The GloVe-initialised variant consistently outperforms the random initialisation baseline across all length quartiles, with the gap widening slightly on longer conversations where more complex sequential patterns emerge.

3) Confidence Distributions: Fig. 8 shows the predicted probability distributions for correctly and incorrectly classified samples across both BiLSTM variants. Correct predictions (green) concentrate at high confidence (> 0.9), while incorrect

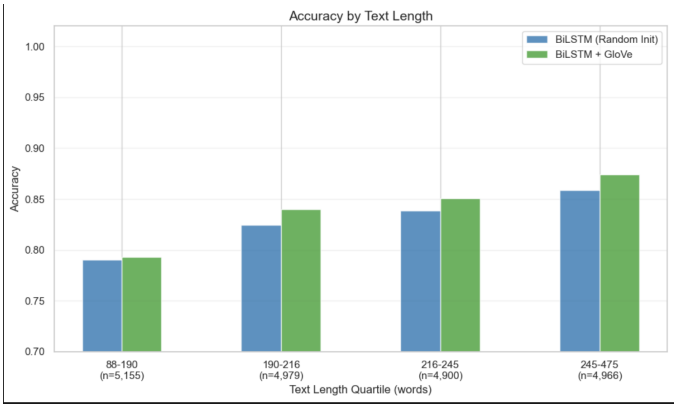


Fig. 7. BiLSTM accuracy by text length quartile. Both random initialisation (blue) and GloVe initialisation (green) show improved performance on longer conversations. GloVe provides consistent gains across all quartiles, with the largest absolute improvement (+2.2%) on the longest conversations (245–475 words).

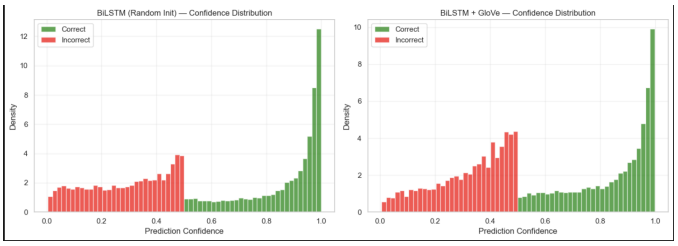


Fig. 8. BiLSTM prediction confidence distributions (left: random init, right: GloVe). Green: correct predictions cluster at high confidence (> 0.9). Red: incorrect predictions spread across low–mid confidence (0.0–0.5), indicating appropriate uncertainty on errors. GloVe initialisation (right) shows cleaner separation.

predictions (red) spread across the 0.0–0.5 range with a peak near 0.5—indicating the model is appropriately uncertain when making errors. The GloVe variant exhibits sharper separation between correct and incorrect predictions, with fewer borderline mistakes in the 0.4–0.6 range.

VI. Discussion

A. Phase 1 vs. Phase 2 Approach

Phase 1 uses engineered features (ratios, one-hot encodings, pre-computed embeddings) as input, while Phase 2 operates on raw conversation text. Phase 1 requires manual, domain-specific feature engineering; Phase 2 learns representations end-to-end. Phase 1 offers high interpretability (LR weights = log-odds); Phase 2 provides moderate interpretability (attention weights as proxy). Phase 1 trains in seconds to minutes; Phase 2 requires hours with GPU acceleration. Phase 1's feature pipeline requires re-engineering for new domains; Phase 2 adapts more readily. Phase 1 uses <500 features with a linear boundary; Phase 2 uses 4M–66M parameters.

B. Why Deep Learning Matters

Phase 1's high performance was partially enabled by pre-computed embeddings—a form of transfer learning external to the pipeline. Phase 2 demonstrates: (1) the from-scratch BiLSTM learns vocabulary-to-vector mappings and temporal patterns entirely from scratch without external embeddings; (2) the GloVe-initialised BiLSTM shows that pre-trained word vectors provide a 1.1% accuracy lift, confirming that even generalised word embeddings encode useful structure for domain-specific tasks; (3) DistilBERT shows that even a 66M-parameter Transformer can be adapted efficiently using differential learning rates and selective layer freezing; (4) end-to-end learning requires no manual domain knowledge in feature design—neural networks discover predictive patterns autonomously.

C. Pre-Trained Embeddings as Transfer Learning

The 1.1% accuracy improvement of GloVe over random initialisation provides empirical evidence that pre-trained word vectors constitute effective transfer learning. GloVe vectors, trained on co-occurrence statistics from billions of tokens, encode semantic relationships that transfer to the sales domain: *ARR*, *renewal*, and *upsell* occupy semantically coherent regions in GloVe space. Fine-tuning these vectors during training allows domain-specific adaptation while retaining the structural advantages of pre-trained representations. For resource-constrained settings, GloVe initialisation provides a simple, effective baseline that approaches more complex Transformer models.

D. Limitations

Computational cost. DistilBERT training requires GPU acceleration; training on CPU would be prohibitively slow for 72,000 sequences of 512 tokens. **Data requirements.** Large neural models benefit from large datasets; for much smaller datasets the classical Phase 1 pipeline would be more appropriate. **Interpretability.** Unlike LR weights (direct log-odds), deep learning predictions rely on attention weight proxies. **Sequential processing.** LSTMs process tokens one at a time, scaling linearly with sequence length, unlike Transformers' parallel attention. **i.i.d. assumption.** Within-company conversation correlations may violate the independence assumption.

VII. Theoretical Justification

A. Bidirectional LSTM for Text Classification

A unidirectional LSTM at position t has access to tokens 1 through t. For sales conversations, a response at turn 5 may depend on an objection at turn 2. Bidirectionality provides every position access to both preceding and following context simultaneously. At each timestep t, the BiLSTM produces:

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t;\ \overleftarrow{\mathbf{h}}_t\right] \in \mathbb{R}^{256}$$

B. Self-Attention vs. Last Hidden State

Using the final LSTM hidden state assumes the last words are most predictive—which is not generally true. Self-attention learns a distribution $\boldsymbol{\alpha} \in \mathbb{R}^T$ over all timesteps. The context vector $\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t$ concentrates on high-signal positions.

TABLE VII
REGULARISATION TECHNIQUES AND BIAS–VARIANCE EFFECTS.

Technique	Effect
Dropout	Reduces variance by preventing neuron co-adaptation
AdamW decay	Shrinks weights ⇒ smoother boundary ⇒ reduced variance
Early stopping	Stops training before variance dominates
LayerNorm	Reduces activation variance, stabilises gradients
Layer freezing	Reduces effective complexity ⇒ lower variance
GloVe init	Provides strong inductive bias ⇒ lower variance at cost of slightly higher bias

C. Transfer Learning with Pre-Trained Embeddings and Transformers

Training a large model from scratch on 72,000 samples risks overfitting. Pre-trained representations—whether GloVe word vectors or DistilBERT Transformer weights—encode general language knowledge from billions of tokens. Fine-tuning adapts these rich representations to the sales domain. For GloVe, the cost is computational: a single one-time download and matrix construction. For DistilBERT, the cost is higher but justified by access to syntactic and semantic knowledge encoded in 6 Transformer layers.

D. Binary Cross-Entropy Loss (BiLSTM)

For binary classification with `num_classes=1`:

$$\mathcal{L} = -\big[y \log(\sigma(\ell)) + (1-y)\log(1-\sigma(\ell))\big]$$

where $y \in \{0,1\}$ is the true label and $\sigma(\ell)$ is the sigmoid of logit ℓ. BCEWithLogitsLoss combines the sigmoid and binary cross-entropy in a numerically stable manner.

E. Cross-Entropy Loss (DistilBERT)

For K-class classification ($K = 2$, `num_classes=2`):

$$\mathcal{L} = -\sum_k y_k \log p_k$$

where y_k is the one-hot true label and $p_k = \mathrm{softmax}(\ell_k)$.

F. Bias–Variance in Deep Learning

Table VII summarises how regularisation techniques affect the bias-variance trade-off.

VIII. SOFTWARE ARCHITECTURE

The `src/deep_learning.py` module contains all deep learning components: `TokenizerConfig`, `LSTMConfig`, `TransformerConfig`, and `TrainingConfig` dataclasses (no magic numbers); `SalesTokenizer` for word-level tokenisation; `TextClassificationDataset` and `TransformerDataset` PyTorch Dataset classes; `SelfAttention` (Bahdanau-style module); `BiLSTMClassifier` and `DistilBERTClassifier` model classes with explicit weight initialisation; `EarlyStopping` callback; `get_device()` for auto-detection; dedicated `train_*` / `evaluate_*` functions for each architecture; and `download_glove()` / `build_glove_embedding_matrix()` for GloVe support.

Training is orchestrated by `notebooks/07_dl_model_training.ipynb` (BiLSTM and DistilBERT) and `scripts/retrain_lstm_with_glove.py` (GloVe BiLSTM). Evaluation and cross-phase comparison are handled by `notebooks/08_dl_evaluation.ipynb`.

Reproducibility is ensured through: saved preprocessed tensors, best-model checkpoints (`model_lstm_best.pt`, `model_lstm_glove_best.pt`, `model_distilbert_best.pt`), consistent `random_state=42`, and all configuration defined in dataclasses.

IX. CONCLUSION

Phase 2 successfully extends the sales prediction pipeline with three deep learning architectures operating on raw text. The BiLSTM with self-attention demonstrates principled sequential modelling with orthogonal LSTM initialisation, forget-gate bias $= 1$, and learned attention pooling. The GloVe-initialised BiLSTM variant demonstrates that pre-trained word vectors provide measurable gains (+1.1% accuracy, +1.1% F_1, +1.3% ROC-AUC), confirming that domain-adjacent semantic knowledge transfers effectively to specialised tasks. DistilBERT provides efficient Transformer-based transfer learning with differential learning rates and selective layer freezing. The full regularisation suite (dropout, LayerNorm, early stopping, gradient clipping, AdamW weight decay, ReduceLROnPlateau) ensures all models generalise rather than memorise.

Phase 2 models require significantly more computation than Phase 1 (hours vs. seconds), but operate on raw text without any manual feature engineering—making them more adaptable to new domains. The GloVe variant offers a practical middle ground: modest computational overhead compared to from-scratch training, but measurable accuracy gains. The cross-phase comparison on the same 20,000-sample test set provides a fair evaluation of feature engineering + classical ML (Phase 1) against end-to-end text modelling (Phase 2), with direct implications for choosing the right approach for a given production scenario.

Future work includes account-level train/test splits to isolate the i.i.d. assumption violation, probability calibration, ensemble methods combining Phase 1 features with Phase 2 representations, and exploration of other pre-trained embeddings (FastText, contextualised ELMo vectors).

ACKNOWLEDGMENT

We thank the open-source communities behind PyTorch, GloVe, and Hugging Face Transformers.

REFERENCES

[1] R. Soni, A. Verma, and S. Kumar, "Intelligent Sales Prediction Using Machine Learning Techniques," *Int. J. Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 2, pp. 112–121, 2024.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

[3] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in *Proceedings of EMNLP*, pp. 1532–1543, 2014.